mra.py 中图重构模块及其联合训练机制说明手册

2026-04-10

摘要

本文面向仓库当前版本的 `mra.py` 及其直接依赖文件 `graph_gcn_reconstruction.py`，系统说明其中**图重构模块**在多采样率异常检测框架中的代码入口、训练入口、数学原理、设计思路与实现细节。文档只聚焦图重构及其与门控插补器、联合损失、训练集统计量和测试集推理之间的关系，其他模块只在必要的耦合处简要出现。报告重点回答七个问题：第一，图模块到底从哪里进入主程序；第二，真正发生参数更新的训练入口在哪里；第三，图模块内部如何从缺失时序构造邻接矩阵并进行图卷积重构；第四，单个 batch 内部的数据张量如何流动、形状如何变化；第五，五个损失项分别更新哪些参数；第六，训练阶段学到的参数和统计量如何在测试阶段发挥作用；第七，这套实现的工程优点、假设前提和局限在哪里。文中所有结论均以代码实现为依据，并辅以一次真实 batch 的前向和自动求导检查结果，以避免仅停留在静态阅读层面。

关键词： 图学习，图卷积重构，多采样率，缺失感知插补，联合训练，异常检测

目录

1 文档范围与核心结论 **3**
1.1 分析范围 . . . 3
1.2 先给结论 . . . 3

2 代码入口、训练入口与模块边界 **4**
2.1 入口总览 . . . 4
2.2 必须区分两个“训练入口” . . . 4
2.3 整体调用链 . . . 4

3 数据、窗口与多采样率元数据 **5**
3.1 原始数据与掩码 . . . 5
3.2 只用观测值拟合的标准化器 . . . 5
3.3 采样率类别与推断步长 . . . 5
3.4 窗口化后的真实形状 . . . 6

4 图重构模块的输入准备：为什么先做 PreImpute **6**
4.1 训练时存在两种缺失掩码 . . . 6
4.2 模型输入并不是原始零填充窗口 . . . 6
4.3 PreImpute 的数学含义 . . . 7
4.4 为什么图模块也要使用种子序列 . . . 7

5 图学习器的数学原理与代码实现 7
5.1 输入输出定义 7
5.2 从时序窗口压缩出节点动态上下文 8
5.3 静态上下文与隐式坐标 8
5.4 节点表示的构造 8
5.5 边特征、相似度和图先验 9
5.6 邻接矩阵的归一化生成 9
5.7 邻接矩阵张量形状汇总 10

6 GCN 重构器的数学原理与实现 10
6.1 GCNLayer 的核心操作 10
6.2 三层图卷积重构结构 11
6.3 图模块的参数规模 11

7 图模块在双专家门控插补器中的位置 11
7.1 图专家不是最终输出，而是门控专家之一 11
7.2 门控输入输出形状 12

8 联合训练流程、损失分解与参数更新 12
8.1 单个 batch 的训练流程 12
8.2 五项损失的数学形式 13
8.3 为什么 detector loss 仍然会更新图模块 13
8.4 一次自动求导检查得到的更新关系 14
8.5 优化器与梯度稳定机制 14

9 训练集学到的参数和统计量，在测试集里如何发挥作用 14
9.1 需要区分“可训练参数”和“训练集估计量” 14
9.2 梯度更新参数及其测试时作用 15
9.3 训练集估计量及其测试时作用 15
9.4 图模块如何影响测试异常分数 15

10 基于真实 batch 的张量流追踪 15
10.1 外层张量 15
10.2 图模块内部张量 16
10.3 与检测器交接后的形状 16

11 设计思路、优点与局限 17
11.1 为什么这样设计 17
11.2 这套图模块的优点 17
11.3 代码层面需要注意的局限 17

12 结论 17

1 文档范围与核心结论

1.1 分析范围

本文只分析以下内容：

- `mra.py` 中图重构模块的接入位置、训练入口、推理入口和评分入口；
- `graph_gcn_reconstruction.py` 中 `GraphLearner`、`GCNLayer`、`GraphGCNReconstructor` 与 `adjacency_balance_loss` 的实现；
- 图模块与 `PreImpute`、门控融合、训练期 holdout 机制、检测器损失之间的交互；
- 训练集上学到的参数和统计量在测试集中的实际作用链条。

以下内容不是本文重点，因此只在必要处带过：

- 频域专家的独立数学推导；
- Transformer 检测器的完整理论；
- 后处理模块 EWAF、阈值搜索和分类指标的统计学细节。

1.2 先给结论

结合代码、真实数据维度检查和单 batch 梯度验证，可以先得出八条关键结论：

- 在 `mra.py` 的真实执行路径中，图重构模块的前向入口是 `TwoExpertGatedImputer.forward`，更具体地说是其中的 `self.gcn(x_seed, model_missing_mask)`。
- 图重构模块自身的独立训练脚本位于 `graph_gcn_reconstruction.py`，但当用户运行 `mra.py` 时，真正生效的训练入口是 `mra.py` 中的 `train_model`，此时图模块与频域专家、门控网络和 Transformer 检测器一起做**单优化器联合训练**。
- 图学习器不是直接从原始时间序列逐点生成边，而是先把每个变量在窗口内压缩成四维动态描述子：均值、标准差、最近观测值、缺失率，再与可学习静态上下文拼接生成节点表示。
- 邻接矩阵由三部分共同决定：边 MLP 打分、节点表示余弦相似度、由可学习隐式坐标生成的先验邻接；之后再叠加自环偏置、温度缩放、按行 softmax、对称化和再归一化。
- 这里的 GCN 不是做时间维卷积，而是**在每个时间步上沿变量维度做消息传递**。因此图模块建模的是变量间依赖，而不是时间上的自回归依赖。
- 图模块不只被 `gcn_loss` 和 `graph_loss` 更新，还会被 `fusion_loss` 和 `detector_loss` 间接更新。尤其是 `detector_loss` 虽然目标张量做了 `detach`，但依然会沿检测器输入路径反向更新插补器。
- 训练集不仅学出神经网络参数，还估计出若干**非梯度统计量**，包括标准化均值方差、采样率类别 `rate_id`、采样步长 `stride`、训练分数归一化参数以及阈值，这些量都会在测试阶段继续发挥作用。
- 当前默认数据下，训练集和测试集均为 4100×18，窗口长度为 50，图邻接矩阵形状为 $B\times18\times18$，门控张量形状为 $B\times50\times18\times2$。

2 代码入口、训练入口与模块边界

2.1 入口总览

位置	代码定位	作用
主程序入口	mra.py :831--1098	负责解析参数、读取训练/测试数据、标准化、窗口化、构建模型、训练、补全、评分和保存结果。
联合训练入口	mra.py :548--621	真实运行 mra.py 时发生参数更新的入口；图模块在这里与频域专家、门控和检测器一起被 AdamW 更新。
图模块接入点	mra.py :270--340	TwoExpertGatedImputer 内部实例化 GraphGCNReconstructor，并把图专家输出送入门控融合。
联合模型封装	mra.py :457--509	FusionAnomalyModel 把插补器与检测器拼成完整前向链。
联合损失定义	mra.py :512--545	定义 fusion_loss、gcn_loss、freq_loss、detector_loss、graph_loss。
补全推理入口	mra.py :624--662	用训练好的图专家参与全序列补全，并输出最后一个时间步的补全值、门控权重和邻接矩阵样本。
评分统计入口	mra.py :665--722	用图专家与频域专家的输出计算专家分歧度、门控熵和检测器分数。
图学习器定义	graph_gcn_ reconstruction.py :241--324	根据窗口级节点统计量构造邻接矩阵。
图卷积重构器定义	graph_gcn_ reconstruction.py :339--362	在学习到的邻接矩阵上执行三层变量维图卷积，输出图专家重构结果。
图模块独立训练入口	graph_gcn_ reconstruction.py :380--432	从 mra.py 抽出的单模块训练脚本，用于独立验证图重构器，但不是 mra.py 的主训练入口。

2.2 必须区分两个“训练入口”

从文件结构看，图模块有两个训练入口，但语义不同：

1. **独立入口**：graph_gcn_reconstruction.py:380-432。这个入口只训练图模块本身，损失是重构误差加图正则，主要用于拆分实验。
2. **联合入口**：mra.py:548-621。这是运行 python mra.py 时真正使用的入口，图模块的参数会与频域专家、门控层和检测器共同优化。

因此，如果问题是“mra.py 中图模块如何训练”，答案必须以第二条为主；第一条更接近参考脚本。

2.3 整体调用链

按代码调用顺序，mra.py 中与图模块相关的主链路可写成

$$\begin{aligned}&\texttt{main} \to \texttt{train_model} \to \texttt{FusionAnomalyModel.forward}\\&\quad \to \texttt{TwoExpertGatedImputer.forward} \to \texttt{GraphGCNReconstructor.forward} \to \texttt{GraphLearner.forward}.\end{aligned} \tag{1}$$

其中最关键的一步发生在

```
x_gcn, adjacency = self.gcn(x_seed, model_missing_mask),
```
(2)

也就是图专家并不直接吃原始零填充窗口，而是吃 `PreImpute` 生成的种子序列 $\boldsymbol{X}_{\text{seed}}$。

3 数据、窗口与多采样率元数据

3.1 原始数据与掩码

`load_csv_glob_with_mask` 会读取 CSV 文件，并把 `NaN` 转成结构性缺失掩码。对当前仓库默认数据：

$$\boldsymbol{X}_{\text{raw}}^{\text{train}} \in \mathbb{R}^{4100\times 18}, \tag{3}$$

$$\boldsymbol{X}_{\text{raw}}^{\text{test}} \in \mathbb{R}^{4100\times 18}, \tag{4}$$

$$\boldsymbol{M}_s^{\text{train}}, \boldsymbol{M}_s^{\text{test}} \in \{0,1\}^{4100\times 18}. \tag{5}$$

用真实数据检查得到训练集与测试集缺失率都约为

$$\text{mean}(\boldsymbol{M}_s) \approx 0.4888618. \tag{6}$$

这里 $M_{s,t,n}=1$ 表示第 t 个时间步、第 n 个变量在原始数据中缺失。

3.2 只用观测值拟合的标准化器

`ObservedStandardScaler` 的设计非常重要。它不是对全列直接求均值方差，而是只用观测值估计 μ_n 与 σ_n，再把缺失点先用 μ_n 填上后做标准化。因此标准化后的输入满足

$$X_{t,n} = \begin{cases} \dfrac{X_{t,n}^{\text{raw}} - \mu_n}{\sigma_n}, & M_{s,t,n}=0, \\ 0, & M_{s,t,n}=1. \end{cases} \tag{7}$$

这有两个直接后果：

- 缺失位置数值上变成了零；
- 但零并不代表真实观测值为零，而必须结合掩码解释。

3.3 采样率类别与推断步长

`infer_rate_metadata` 根据训练集缺失率推断每列变量的观测率，再取倒数近似采样步长：

$$r_n = 1 - \frac{1}{T_{\text{all}}}\sum_{t=1}^{T_{\text{all}}} M_{s,t,n}, \qquad s_n = \text{round}\left(\frac{1}{\max(r_n, 10^{-6})}\right). \tag{8}$$

当前默认数据上得到

$$\boldsymbol{stride} = [1,1,1,1,1,1,3,3,3,3,3,3,5,5,5,5,5,5], \tag{9}$$

$$\boldsymbol{rate_id} = [0,0,0,0,0,0,1,1,1,1,1,1,2,2,2,2,2,2]. \tag{10}$$

这意味着 18 个变量被分成三组：

- 第 1–6 列：高频组，步长约为 1；
- 第 7–12 列：中频组，步长约为 3；
- 第 13–18 列：低频组，步长约为 5。

图模块本身不直接使用 `stride`，但门控层和检测器会使用 `rate_id/stride`，因此这些训练集推断出的元数据会间接影响图模块在联合系统中的作用。

3.4 窗口化后的真实形状

`mra.py` 同时构建三类窗口：

- `build_windows` 生成训练插补窗口和完整序列补全窗口，支持前端补齐；
- `build_standard_windows` 生成训练评分窗口；
- `build_prompt_test_windows` 生成测试评分窗口及其固定标签。

默认参数 `seq_len=50`、`stride=1` 下，真实形状为

$$\boldsymbol{W}_{\text{imp}}^{\text{train}} \in \mathbb{R}^{4100\times 50\times 18}, \tag{11}$$

$$\boldsymbol{W}_{\text{imp}}^{\text{test}} \in \mathbb{R}^{4100\times 50\times 18}, \tag{12}$$

$$\boldsymbol{W}_{\text{eval}}^{\text{train}} \in \mathbb{R}^{4001\times 50\times 18}, \tag{13}$$

$$\boldsymbol{W}_{\text{eval}}^{\text{test}} \in \mathbb{R}^{4000\times 50\times 18}. \tag{14}$$

测试集标签长度为 4000，其中前 2000 个窗口被标为正常，后 2000 个窗口被标为异常。这一划分来自 `utils/methods/windowing.py:6-10,133-167` 的常量配置。

4 图重构模块的输入准备：为什么先做 PreImpute

4.1 训练时存在两种缺失掩码

在训练 batch 内部，代码同时处理两种掩码：

- 结构性缺失掩码 $\boldsymbol{M}_s$：原始数据本来就缺失的位置；
- 训练期额外遮挡掩码 $\boldsymbol{M}_h$：从观测值中按比例随机抽掉的位置，用于自监督重构。

实际送入模型的掩码为

$$\boldsymbol{M} = \max(\boldsymbol{M}_s, \boldsymbol{M}_h), \tag{15}$$

其中默认 `holdout_ratio=0.15`。代码还额外保证：如果一个样本窗口中有观测值，但随机采样恰好没有抽到任何 holdout 点，则强制再抽一个位置，避免该样本对重构损失没有监督信号。

4.2 模型输入并不是原始零填充窗口

训练时模型先构造

$$\boldsymbol{X}_{\text{input}} = \boldsymbol{X}_{\text{true}} \odot (1 - \boldsymbol{M}), \tag{16}$$

也就是把结构性缺失和 holdout 缺失统一置零。随后并不直接把 $\boldsymbol{X}_{\text{input}}$ 送给图模块，而是先做

$$\boldsymbol{X}_{\text{seed}} = \text{PreImpute}(\boldsymbol{X}_{\text{input}}, \boldsymbol{M}). \tag{17}$$

4.3 PreImpute 的数学含义

`PreImpute.fill` 对每个变量沿时间轴做一次正向填充和一次反向填充。若记第 n 个变量在窗口内第 t 个位置的输入为 $x_{t,n}$，则：

- 正向填充值 $f^{\rightarrow}_{t,n}$ 表示当前时刻之前最近一次可见观测；
- 反向填充值 $f^{\leftarrow}_{t,n}$ 表示当前时刻之后最近一次可见观测；
- 若两侧都有值，则种子取平均；
- 若只有单侧有值，则取该侧；
- 若两侧都没有值，则仍然保留零。

可以写成分段形式

$$x^{\text{seed}}_{t,n} = \begin{cases} x_{t,n}, & M_{t,n} = 0, \\ \dfrac{f^{\rightarrow}_{t,n} + f^{\leftarrow}_{t,n}}{2}, & M_{t,n} = 1 \text{ 且两侧均可用}, \\ f^{\rightarrow}_{t,n}, & M_{t,n} = 1 \text{ 且仅前向可用}, \\ f^{\leftarrow}_{t,n}, & M_{t,n} = 1 \text{ 且仅后向可用}, \\ 0, & M_{t,n} = 1 \text{ 且两侧均不可用}. \end{cases} \tag{18}$$

4.4 为什么图模块也要使用种子序列

图模块本质上依赖窗口统计量来构造节点表征。如果直接对大段零填充输入做均值、标准差和最近观测统计，很多节点描述会被“人为零值”污染。先做双向种子插补有两个作用：

- 缓和高缺失率变量的统计量偏移；
- 让图学习器更像是在分析“近似完整但带不确定性”的局部轨迹，而不是大量人工零块。

5 图学习器的数学原理与代码实现

5.1 输入输出定义

图学习器接收

$$\boldsymbol{X}_{\text{seed}} \in \mathbb{R}^{B\times T\times N}, \qquad \boldsymbol{M} \in \{0,1\}^{B\times T\times N}, \tag{19}$$

输出样本级邻接矩阵

$$\boldsymbol{A} \in \mathbb{R}^{B\times N\times N}. \tag{20}$$

在默认配置下 $T = 50$、$N = 18$。

5.2 从时序窗口压缩出节点动态上下文

`GraphLearner._dynamic_context` 对每个样本、每个变量提取四个量：

$$\mu_{b,n} = \frac{\sum_{t=1}^{T}(1-M_{b,t,n})X_{b,t,n}^{\text{seed}}}{\sum_{t=1}^{T}(1-M_{b,t,n})+\varepsilon}, \tag{21}$$

$$\sigma_{b,n} = \sqrt{\frac{\sum_{t=1}^{T}(1-M_{b,t,n})\left(X_{b,t,n}^{\text{seed}}-\mu_{b,n}\right)^2}{\sum_{t=1}^{T}(1-M_{b,t,n})+\varepsilon}+10^{-6}}, \tag{22}$$

$$\ell_{b,n} = \text{最近一次被观测到的值}, \tag{23}$$

$$\rho_{b,n} = \frac{1}{T}\sum_{t=1}^{T} M_{b,t,n}. \tag{24}$$

于是动态上下文张量为

$$\boldsymbol{D} \in \mathbb{R}^{B\times N\times 4}. \tag{25}$$

文字解释 这四个量并不是随意拼的。它们分别回答了四个问题：这个变量在当前窗口内平均水平是多少、波动有多大、最近一次有效状态是什么、缺失得有多严重。图学习器不逐点处理整段序列，而是先把每个变量压缩成这样一个缺失感知的统计摘要，再基于摘要推断变量间关系。

5.3 静态上下文与隐式坐标

除了动态统计量，代码还给每个变量分配了两组可学习参数：

- 静态上下文 $\boldsymbol{S} \in \mathbb{R}^{N\times 8}$；
- 隐式坐标 $\boldsymbol{C} \in \mathbb{R}^{N\times 8}$。

其中静态上下文用于参与节点编码，隐式坐标用于构造图先验。要强调的是，$\boldsymbol{C}$ 不是传感器真实物理坐标，而是模型训练中学出来的潜在几何位置。

5.4 节点表示的构造

对每个样本 b、变量 n，先把动态上下文和静态上下文拼接：

$$\boldsymbol{z}_{b,n} = [\boldsymbol{d}_{b,n}; \boldsymbol{s}_n] \in \mathbb{R}^{12}. \tag{26}$$

然后通过两层 MLP：

$$\boldsymbol{h}_{b,n} = \phi_{\text{node}}(\boldsymbol{z}_{b,n}) \in \mathbb{R}^{H}, \tag{27}$$

其中默认 $H = 32$。因此节点表示张量形状为

$$\boldsymbol{H} \in \mathbb{R}^{B\times N\times 32}. \tag{28}$$

代码随后又进行 L_2 归一化，得到

$$\tilde{\boldsymbol{h}}_{b,n} = \frac{\boldsymbol{h}_{b,n}}{\|\boldsymbol{h}_{b,n}\|_2}. \tag{29}$$

5.5 边特征、相似度和图先验

对于每一对节点 (i,j)，代码构造如下拼接特征：

$$\boldsymbol{p}_{b,i,j} = [\boldsymbol{h}_{b,i}; \boldsymbol{h}_{b,j}; |\boldsymbol{h}_{b,i} - \boldsymbol{h}_{b,j}|; \boldsymbol{h}_{b,i} \odot \boldsymbol{h}_{b,j}] \in \mathbb{R}^{4H}. \tag{30}$$

当 $H = 32$ 时，边特征维度为 128，因此

$$\boldsymbol{P} \in \mathbb{R}^{B\times N\times N\times 128}. \tag{31}$$

边 MLP 给出原始边打分

$$e^{\text{mlp}}_{b,i,j} = \phi_{\text{edge}}(\boldsymbol{p}_{b,i,j}). \tag{32}$$

与此同时，代码还计算节点表示的余弦相似度

$$s_{b,i,j} = \tilde{\boldsymbol{h}}_{b,i}^{\top} \tilde{\boldsymbol{h}}_{b,j}, \tag{33}$$

以及基于隐式坐标的图先验

$$p_{i,j} = \frac{1}{1 + \|\boldsymbol{c}_i - \boldsymbol{c}_j\|_2}, \qquad i \neq j. \tag{34}$$

文字解释 这里的设计思路很明确：边 MLP 负责学习复杂的非线性边关系；余弦相似度提供一个几何上直接的相似项；坐标先验提供一个样本无关的“默认连边趋势”。三者叠加以后，邻接矩阵不完全依赖某一条证据，从而稳定性更强。

5.6 邻接矩阵的归一化生成

代码中的最终 logit 形式为

$$\ell_{b,i,j} = e^{\text{mlp}}_{b,i,j} + \alpha\, p_{i,j} + \beta\, s_{b,i,j}, \tag{35}$$

其中

$$\alpha = \text{softplus}(\texttt{prior_strength}), \qquad \beta = \text{softplus}(\texttt{similarity_strength}). \tag{36}$$

对角线上再额外加入可学习自环偏置

$$\ell_{b,i,i} \leftarrow \ell_{b,i,i} + \text{softplus}(\texttt{self_loop_logit}), \tag{37}$$

并用温度参数

$$\tau = \text{softplus}(\texttt{temperature}) + 10^{-4} \tag{38}$$

做缩放，随后按行 softmax：

$$\bar{A}_{b,i,j} = \frac{\exp(\ell_{b,i,j}/\tau)}{\sum_{m=1}^{N} \exp(\ell_{b,i,m}/\tau)}. \tag{39}$$

接着代码又做两步处理：

$$\tilde{A}_{b,i,j} = \frac{1}{2}\left(\bar{A}_{b,i,j} + \bar{A}_{b,j,i}\right), \tag{40}$$

$$A_{b,i,j} = \frac{\tilde{A}_{b,i,j}}{\sum_{m=1}^{N} \tilde{A}_{b,i,m}}. \tag{41}$$

非常重要的实现细节 很多人看到“对称化”会下意识认为最终邻接矩阵一定严格对称，但这里并非如此。原因是代码先对称化，再做**按行归一化**。最后得到的 $\boldsymbol{A}$ 满足每一行和为 1，但一般只保持**近似对称**而非严格对称。一次真实 batch 检查得到：

- 行和均值约为 1.0；
- 行和标准差约为 6.06×10^{-8}；
- 初始对角均值约为 0.0449；
- 平均对称误差约为 2.59×10^{-3}。

这说明该实现更准确的说法是：**行随机、近似对称的样本级邻接矩阵**。

5.7 邻接矩阵张量形状汇总

对于一个 batch，图学习器内部各主要张量形状为：

$$\boldsymbol{X}_{\text{seed}} : [B, 50, 18], \tag{42}$$

$$\boldsymbol{D} : [B, 18, 4], \tag{43}$$

$$\boldsymbol{S}_{\text{expand}} : [B, 18, 8], \tag{44}$$

$$\boldsymbol{Z} : [B, 18, 12], \tag{45}$$

$$\boldsymbol{H} : [B, 18, 32], \tag{46}$$

$$\boldsymbol{P} : [B, 18, 18, 128], \tag{47}$$

$$\boldsymbol{A} : [B, 18, 18]. \tag{48}$$

6 GCN 重构器的数学原理与实现

6.1 GCNLayer 的核心操作

`GCNLayer.forward` 接收

$$\boldsymbol{X} \in \mathbb{R}^{B \times T \times N \times d_{\text{in}}}, \qquad \boldsymbol{A} \in \mathbb{R}^{B \times N \times N}, \tag{49}$$

先在特征维上做线性映射，再沿变量维度执行消息传递：

$$\boldsymbol{Y}_{b,t,n,:} = \sum_{m=1}^{N} A_{b,n,m} \, \boldsymbol{W} \boldsymbol{X}_{b,t,m,:}. \tag{50}$$

对应代码是

$$\texttt{torch.einsum("bnm,bsmd->bsnd", adj, x)}. \tag{51}$$

文字解释 这里的图卷积只在节点维 N 上传播，不在时间维 T 上卷积。也就是说，对每个时间步 t 都会使用同一张样本级邻接矩阵 $\boldsymbol{A}_b$ 来混合 18 个变量的信息。时间建模主要由窗口统计和后续检测器负责，图模块负责的是“同一时间截面上变量之间该如何借信息”。

6.2 三层图卷积重构结构

`GraphGCNReconstructor` 的前向流程为

$$\boldsymbol{H}^{(1)} = \mathrm{Dropout}(\mathrm{LayerNorm}(\mathrm{ReLU}(\mathrm{GCN}_1(\boldsymbol{X}_{\mathrm{seed}}, \boldsymbol{A})))), \tag{52}$$

$$\boldsymbol{H}^{(2)} = \mathrm{Dropout}\big(\mathrm{LayerNorm}\big(\mathrm{ReLU}\big(\mathrm{GCN}_2(\boldsymbol{H}^{(1)}, \boldsymbol{A})\big)\big)\big), \tag{53}$$

$$\boldsymbol{X}_{\mathrm{gcn}} = \mathrm{GCN}_3(\boldsymbol{H}^{(2)}, \boldsymbol{A}). \tag{54}$$

默认维度为：

$$\boldsymbol{X}_{\mathrm{seed}} : [B, 50, 18], \tag{55}$$

$$\boldsymbol{X}^{+}_{\mathrm{seed}} = \boldsymbol{X}_{\mathrm{seed}}\texttt{.unsqueeze(-1)} : [B, 50, 18, 1], \tag{56}$$

$$\boldsymbol{H}^{(1)} : [B, 50, 18, 32], \tag{57}$$

$$\boldsymbol{H}^{(2)} : [B, 50, 18, 32], \tag{58}$$

$$\boldsymbol{X}_{\mathrm{gcn}} : [B, 50, 18]. \tag{59}$$

6.3 图模块的参数规模

在默认配置下，图专家相关参数量为：

- 图学习器 `gcn.graph`：5925；
- GCN 三层线性与两层 LayerNorm：1281；
- 图专家总计：7206。

若把整个插补器都算上，还包括：

- 频域专家：24314；
- 采样率嵌入：24；
- 门控 MLP：5250；
- 插补器总参数：36794；
- 全模型总参数：693260。

7 图模块在双专家门控插补器中的位置

7.1 图专家不是最终输出，而是门控专家之一

`TwoExpertGatedImputer` 同时产生两路专家输出：

$$\boldsymbol{X}_{\mathrm{gcn}} = f_{\mathrm{gcn}}(\boldsymbol{X}_{\mathrm{seed}}, \boldsymbol{M}), \qquad \boldsymbol{X}_{\mathrm{freq}} = f_{\mathrm{freq}}(\boldsymbol{X}_{\mathrm{seed}}). \tag{60}$$

随后门控网络使用六类局部特征：

$$[\boldsymbol{X}_{\mathrm{seed}}, \boldsymbol{M}, \boldsymbol{X}_{\mathrm{gcn}}, \boldsymbol{X}_{\mathrm{freq}}, \boldsymbol{X}_{\mathrm{gcn}} - \boldsymbol{X}_{\mathrm{seed}}, \boldsymbol{X}_{\mathrm{freq}} - \boldsymbol{X}_{\mathrm{seed}}], \tag{61}$$

再拼接采样率嵌入，得到门控输入。

7.2 门控输入输出形状

默认 `rate_embed_dim=8`，因此：

- 六类标量特征堆叠后是 $[B, T, N, 6]$；
- 采样率嵌入扩展后是 $[B, T, N, 8]$；
- 门控输入是 $[B, T, N, 14]$；
- 门控 logits 是 $[B, T, N, 2]$；
- softmax 后门控权重 $\mathbf{\Pi}$ 也是 $[B, T, N, 2]$。

于是最终插补值为

$$\boldsymbol{X}_{\text{imp}} = \Pi_{:,:,:,0} \odot \boldsymbol{X}_{\text{gcn}} + \Pi_{:,:,:,1} \odot \boldsymbol{X}_{\text{freq}}. \tag{62}$$

完整窗口输出为

$$\boldsymbol{X}_{\text{complete}} = \boldsymbol{M} \odot \boldsymbol{X}_{\text{imp}} + (1 - \boldsymbol{M}) \odot \boldsymbol{X}_{\text{input}}. \tag{63}$$

文字解释 这表示图专家只在“模型认为缺失的位置”上负责补值；对于未缺失的位置，系统直接保留原始输入，不让专家覆盖真实观测。这样可以避免补全网络把原本正确的观测又改坏。

8 联合训练流程、损失分解与参数更新

8.1 单个 batch 的训练流程

设一个训练 batch 的真实标准化窗口为

$$\boldsymbol{X}_{\text{true}} \in \mathbb{R}^{B \times 50 \times 18}, \tag{64}$$

则 `mra.py` 中一次前向和反向传播可概括为：

1. 从 DataLoader 取出 $(\boldsymbol{X}_{\text{true}}, \boldsymbol{M}_s)$；
2. 在观测位置采样 holdout 掩码 $\boldsymbol{M}_h$；
3. 得到模型缺失掩码 $\boldsymbol{M} = \max(\boldsymbol{M}_s, \boldsymbol{M}_h)$；
4. 构造零填充输入 $\boldsymbol{X}_{\text{input}}$；
5. 经 `PreImpute` 得到 $\boldsymbol{X}_{\text{seed}}$；
6. 图专家输出 $\boldsymbol{X}_{\text{gcn}}$ 与邻接矩阵 $\boldsymbol{A}$；
7. 频域专家输出 $\boldsymbol{X}_{\text{freq}}$；
8. 门控融合得到 $\boldsymbol{X}_{\text{imp}}$ 与 $\boldsymbol{X}_{\text{complete}}$；
9. 检测器以 $\boldsymbol{X}_{\text{complete}}$ 为输入，输出 $\boldsymbol{X}_{\text{det}}$；
10. 计算五项损失并加权求和；
11. `optimizer.zero_grad()`；

12. `total_loss.backward()`;
13. `clip_grad_norm_(..., 1.0)`;
14. `optimizer.step()`。

8.2 五项损失的数学形式

1. 融合插补损失

$$\mathcal{L}_{\text{fusion}} = \frac{\|(\boldsymbol{X}_{\text{imp}} - \boldsymbol{X}_{\text{true}}) \odot \boldsymbol{M}_h\|_F^2}{\|\boldsymbol{M}_h\|_1 + \varepsilon}. \tag{65}$$

2. 图专家损失

$$\mathcal{L}_{\text{gcn}} = \frac{\|(\boldsymbol{X}_{\text{gcn}} - \boldsymbol{X}_{\text{true}}) \odot \boldsymbol{M}_h\|_F^2}{\|\boldsymbol{M}_h\|_1 + \varepsilon}. \tag{66}$$

3. 频域专家损失

$$\mathcal{L}_{\text{freq}} = \frac{\|(\boldsymbol{X}_{\text{freq}} - \boldsymbol{X}_{\text{true}}) \odot \boldsymbol{M}_h\|_F^2}{\|\boldsymbol{M}_h\|_1 + \varepsilon}. \tag{67}$$

4. 图平衡损失

$$\mathcal{L}_{\text{graph}} = \frac{1}{BN} \sum_{b=1}^{B} \sum_{n=1}^{N} (A_{b,n,n} - \delta)^2, \tag{68}$$

其中默认 $\delta = \texttt{diag_target} = 0.25$。

5. 检测器重构损失

$$\mathcal{L}_{\text{det}} = \frac{\|(f_{\text{det}}(\boldsymbol{X}_{\text{complete}}) - \text{sg}(\boldsymbol{X}_{\text{complete}})) \odot (1 - \boldsymbol{M}_s)\|_F^2}{\|(1 - \boldsymbol{M}_s)\|_1 + \varepsilon}. \tag{69}$$

这里 $\text{sg}(\cdot)$ 表示 stop-gradient，也就是代码中的 `detach`。

总损失 默认权重来自 `LossWeights`：

$$\mathcal{L} = 1.0\mathcal{L}_{\text{fusion}} + 0.4\mathcal{L}_{\text{gcn}} + 0.4\mathcal{L}_{\text{freq}} + 0.5\mathcal{L}_{\text{det}} + 0.1\mathcal{L}_{\text{graph}}. \tag{70}$$

8.3 为什么 detector loss 仍然会更新图模块

这是很多人第一次读代码时容易误判的地方。因为

$$\boldsymbol{X}_{\text{target}} = \text{sg}(\boldsymbol{X}_{\text{complete}}) \tag{71}$$

做了 `detach`，所以**目标分支**不再向插补器回传梯度；但检测器前向输入仍然是 $\boldsymbol{X}_{\text{complete}}$，因此

$$\frac{\partial \mathcal{L}_{\text{det}}}{\partial \boldsymbol{X}_{\text{complete}}} = \left(\frac{\partial f_{\text{det}}(\boldsymbol{X}_{\text{complete}})}{\partial \boldsymbol{X}_{\text{complete}}} \right)^{\top} \cdot \frac{\partial \mathcal{L}_{\text{det}}}{\partial f_{\text{det}}}. \tag{72}$$

因此，检测器损失仍会沿

$$\boldsymbol{X}_{\text{complete}} \to \boldsymbol{X}_{\text{imp}} \to (\boldsymbol{X}_{\text{gcn}}, \boldsymbol{X}_{\text{freq}}, \boldsymbol{\Pi}) \tag{73}$$

这条路径更新图专家、频域专家和门控层。

8.4 一次自动求导检查得到的更新关系

为了避免仅凭经验判断，本文对一个真实 batch 做了分损失反向传播检查。结果表明：

损失项	图学习器	GCN 层	频域专家	门控层	检测器
$\mathcal{L}_{\text{fusion}}$	更新	更新	更新	更新	不更新
$\mathcal{L}_{\text{gcn}}$	更新	更新	不更新	不更新	不更新
$\mathcal{L}_{\text{freq}}$	不更新	不更新	更新	不更新	不更新
$\mathcal{L}_{\text{det}}$	更新	更新	更新	更新	更新
$\mathcal{L}_{\text{graph}}$	更新	不更新	不更新	不更新	不更新

这张表非常关键，因为它说明图模块的训练信号来自四条路径，而不是两条：

- 直接重构监督：$\mathcal{L}_{\text{gcn}}$；
- 邻接结构正则：$\mathcal{L}_{\text{graph}}$；
- 通过门控后的融合重构：$\mathcal{L}_{\text{fusion}}$；
- 通过检测器输入一致性传回来的高层约束：$\mathcal{L}_{\text{det}}$。

8.5 优化器与梯度稳定机制

训练中使用

- 优化器：`AdamW`；
- 学习率：`1e-3`；
- 权重衰减：`1e-4`；
- 梯度裁剪：全模型 ℓ_2 范数裁剪到 1.0。

梯度裁剪对这套结构很重要，因为：

- 邻接矩阵由 softmax 与温度参数控制，梯度可能偏尖锐；
- 图专家、频域专家和检测器共享反向链路，存在多损失叠加；
- 高缺失率变量容易带来更不稳定的误差信号。

9 训练集学到的参数和统计量，在测试集里如何发挥作用

9.1 需要区分“可训练参数”和“训练集估计量”

用户关心“训练集更新的参数和在测试集中的作用”，实际上应该拆成两类：

1. **通过反向传播更新的参数**：神经网络权重和可学习嵌入；
2. **由训练集估计但不参与反向传播的量**：标准化统计量、采样率元数据、训练分数均值方差、阈值等。

9.2 梯度更新参数及其测试时作用

参数组	训练阶段如何更新	测试阶段有什么作用
`imputer.gcn.graph.*`	受 $\mathcal{L}_{\text{fusion}}, \mathcal{L}_{\text{gcn}}, \mathcal{L}_{\text{det}}, \mathcal{L}_{\text{graph}}$ 更新	决定测试窗口的邻接矩阵，进而决定图专家如何跨变量传播信息。
`imputer.gcn.gcn_*` 和 `LayerNorm`	受 $\mathcal{L}_{\text{fusion}}, \mathcal{L}_{\text{gcn}}, \mathcal{L}_{\text{det}}$ 更新	决定图专家输出 $\boldsymbol{X}_{\text{gcn}}$，直接影响补全值、专家分歧分数和最终异常分数。
`imputer.rate_embedding` 与 `imputer.gate`	受 $\mathcal{L}_{\text{fusion}}, \mathcal{L}_{\text{det}}$ 更新	决定测试时每个变量每个时间点更信任图专家还是频域专家。
`detector.*`	只受 $\mathcal{L}_{\text{det}}$ 更新	决定测试集上的检测器重构误差 `detector_score`。

9.3 训练集估计量及其测试时作用

量	来源	测试阶段作用
μ_n, σ_n	`ObservedStandardScaler.fit(train_raw, train_mask)`	用训练集统计量标准化测试集，并在补全结束后做逆变换恢复物理量纲。
`rate_id`、`stride`	`infer_rate_metadata(train_mask)`	决定测试集门控嵌入与检测器的采样相位编码。这里隐含假设：训练集和测试集具有一致的采样结构。
训练分数均值方差	`normalize_scores` 内部对 train stats 求均值、标准差	用训练分数分布去标准化测试分数，保证异常判别是“相对训练正常分布”的偏离。
`shift_score` 的均值方差	`compute_distribution_shift_scores(train, eval)`	用训练补全窗口定义分布偏移基准，再衡量测试补全窗口是否偏离。
阈值 θ	`choose_threshold(train_scores, ...)`	用训练集平滑分数确定异常阈值，再直接作用于测试分数。

9.4 图模块如何影响测试异常分数

图模块在测试阶段至少通过四条路径影响最终结果：

1. 通过 $\boldsymbol{X}_{\text{gcn}}$ 参与门控融合，改变 $\boldsymbol{X}_{\text{complete}}$；
2. 通过 $\boldsymbol{X}_{\text{gcn}}$ 与 $\boldsymbol{X}_{\text{freq}}$ 的差异，直接影响 `disagreement_score`；
3. 通过门控 logits 间接影响 `gate_entropy`，虽然当前最终分数未显式使用门控熵，但它被保存并可用于分析；
4. 通过改变补全后的窗口分布，间接影响 `shift_score` 与检测器输入，从而影响最终融合分数。

10 基于真实 batch 的张量流追踪

10.1 外层张量

对真实数据取 $B = 4$ 的一个 batch，主要张量形状如下：

张量名	形状	说明
$\boldsymbol{X}_{\text{true}}$	$[4, 50, 18]$	真实标准化窗口
$\boldsymbol{M}_s$	$[4, 50, 18]$	结构性缺失掩码
$\boldsymbol{M}_h$	$[4, 50, 18]$	训练期 holdout 掩码
$\boldsymbol{M}$	$[4, 50, 18]$	实际模型缺失掩码
$\boldsymbol{X}_{\text{input}}$	$[4, 50, 18]$	零填充输入
$\boldsymbol{X}_{\text{seed}}$	$[4, 50, 18]$	预插补种子
$\boldsymbol{X}_{\text{gcn}}$	$[4, 50, 18]$	图专家输出
$\boldsymbol{X}_{\text{freq}}$	$[4, 50, 18]$	频域专家输出
$\boldsymbol{X}_{\text{imp}}$	$[4, 50, 18]$	门控融合补值
$\boldsymbol{X}_{\text{complete}}$	$[4, 50, 18]$	完整窗口
$\boldsymbol{\Pi}$	$[4, 50, 18, 2]$	两专家门控权重
$\boldsymbol{A}$	$[4, 18, 18]$	样本级邻接矩阵
$\boldsymbol{X}_{\text{det}}$	$[4, 50, 18]$	检测器重构输出

10.2 图模块内部张量

图模块内部的关键形状进一步细化为：

内部张量	形状	来源
节点动态上下文 $\boldsymbol{D}$	$[4, 18, 4]$	均值、标准差、最近观测、缺失率
扩展静态上下文	$[4, 18, 8]$	可学习参数复制到 batch
节点输入 $\boldsymbol{Z}$	$[4, 18, 12]$	动态 + 静态拼接
节点表示 $\boldsymbol{H}$	$[4, 18, 32]$	节点编码 MLP 输出
成对边特征 $\boldsymbol{P}$	$[4, 18, 18, 128]$	$h_i, h_j, \lvert h_i - h_j \rvert, h_i \odot h_j$
第一层 GCN 隐状态	$[4, 50, 18, 32]$	`gcn_in` 输出
第二层 GCN 隐状态	$[4, 50, 18, 32]$	`gcn_mid` 输出
图重构输出 $\boldsymbol{X}_{\text{gcn}}$	$[4, 50, 18]$	`gcn_out` 输出

10.3 与检测器交接后的形状

虽然本文不展开检测器全部理论，但必须说明图模块输出如何继续流动。默认检测器维度为 128，因此

- `per_variable_dim = max(128 / 8, 8) = 16`；
- 每个变量的 rate-aware 特征维度为 16；
- token 输入维度为 $18 \times 2 + 18 \times 16 = 324$；
- 经过线性投影后进入 $d_{\text{model}} = 128$ 的 Transformer 编码器。

这说明图模块输出不仅决定插补值本身，还改变了后续检测器看到的 token 分布。

11 设计思路、优点与局限

11.1 为什么这样设计

从工程角度看，这个图模块的设计有三层考虑：

- **缺失感知**：先用掩码统计和种子插补，把“缺失模式”变成图学习的一部分，而不是把缺失简单当成零值；
- **样本自适应**：邻接矩阵按窗口动态生成，而不是整套数据共用一张固定图；
- **与多专家融合兼容**：图专家不必单独承担所有补全任务，而是作为一个结构化专家和频域专家竞争/协作。

11.2 这套图模块的优点

- 邻接矩阵由当前窗口统计量驱动，能针对不同工况动态变化；
- 节点动态描述子显式包含缺失率，因此对多采样率数据更友好；
- 图先验、相似度和边 MLP 三条证据并用，稳定性比单一相似度建图更好；
- 图模块在联合训练中不仅追求局部重构精度，也被最终检测任务反向约束。

11.3 代码层面需要注意的局限

- 图学习器只使用窗口级统计量，而不是完整序列动态，因此会丢失细粒度时间模式；
- `adjacency_balance_loss` 只约束对角占比，不约束稀疏性、谱半径或连通结构；
- 对称化后又按行归一化，所以最终邻接矩阵不是严格对称；
- `rate_id` 和 `stride` 是从训练集缺失模式推断的，若测试集采样机制变化，模型可能发生失配；
- 图卷积只沿变量维传播，不显式建模跨时间边，因此图模块本身不是时空图网络。

12 结论

综合来看，`mra.py` 中的图重构模块不是一个孤立的 GCN 插补器，而是多采样率联合异常检测系统中的结构化专家。它的真正价值不只是输出 $\boldsymbol{X}_{\mathrm{gcn}}$，而是通过**样本级图学习**、**门控融合**、**检测器反向约束**三者共同作用，把“变量之间的结构关系”注入到最终补全和异常评分流程中。

若只看局部代码，容易把它理解成“建一张图再做三层 GCN”；但从完整训练链条看，它更准确的定位应当是：**在缺失感知、多专家融合和无监督检测三重目标下，被多损失联合塑形的样本级变量关系建模模块**。这也是本文建议在后续实验中重点关注的地方：不要只比较图专家的单独重构误差，还应同时观察门控权重、专家分歧、邻接矩阵统计和最终异常分数的联动变化。